AI Meets Formal Verification: Practice and Prospects

Liao Zhang

Beneficial AI Foundation

Novermeber 6, 2026

Overview

1. Background: Beneficial AI Foundation and Signal Shot
2. Practice
3. Lessons from Practice
4. Prospects

Self introduction

- A research scientist at beneficial AI foundation.
- Work on AI for formal verification since 2018.
- PhD from University of Innsbruck, Austria
- Focus on AI for math and coding

Self Introduction: Selected Research

AI for Term Rewriting

The first to apply AI for term rewriting and prove the largest number of problems in annual confluence competition.



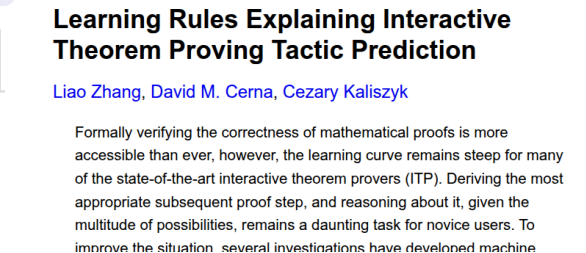
Automated Strategy Invention for Confluence of Term Rewrite Systems

Liao Zhang, Fabian Mitterwallner, Jan Jakubuv, Cezary Kaliszyk

Term rewriting plays a crucial role in software verification and compiler optimization. With dozens of highly parameterizable techniques developed to prove various system properties, automatic term rewriting tools work in an extensive parameter space. This complexity exceeds human capacity for parameter selection, motivating

Interpretable Tactic Prediction

Learning explicit rules that explain tactic predictions in interactive theorem proving.



Learning Rules Explaining Interactive Theorem Proving Tactic Prediction

Liao Zhang, David M. Cerna, Cezary Kaliszyk

Formally verifying the correctness of mathematical proofs is more accessible than ever, however, the learning curve remains steep for many of the state-of-the-art interactive theorem provers (ITP). Deriving the most appropriate subsequent proof step, and reasoning about it, given the multitude of possibilities, remains a daunting task for novice users. To improve the situation, several investigations have developed machine

Self Introduction: Selected Research

Evaluating AI Agents on Property-Based Testing

From documentation to semantic invariants and input-generation strategies that reveal bugs.

PBT-Bench: Benchmarking AI Agents on Property-Based Testing

Lucas Jing, Xinqi Wang, Liao Zhang, Simon S. Du

Existing code benchmarks measure whether an agent can produce any test that reproduces a known bug, or whether it can produce a patch that fixes a described issue. Neither isolates the distinct skill of property-based testing: deriving a semantic invariant from documentation, and then constructing an input-generation strategy precise enough to make a random search reveal the violation.
We introduce PBT-Bench, a benchmark of 100 curated property-based

Roadmap

1. **Background: Beneficial AI Foundation and Signal Shot**
2. **Practice**
3. **Lessons from Practice**
4. **Prospects**

Beneficial AI Foundation

Mission

Keep AI beneficial for humanity through technical AI safety research.

- Founded by Max Tegmark.
- AI is unsafe, what should we do?
- Main approach: AI generates program and verify the correctness and safety of the program. Then use the program to reply to the user instead of using AI.

Connection to practice: cryptographic software provides concrete, demanding verification tasks.



Max Tegmark

Photo: Bengt Oberger / Wikimedia Commons

What Is Signal?

- **Signal the app:** a messaging app for private conversations, including text messages, group chats, and voice/video calls.
- **End-to-end encryption:** messages are encrypted on the sender's device and decrypted on the recipient's device; the service does not have the keys to read them. Even if a hacker attacks the server, he cannot get messages from all users.
- **Signal Protocol:** the cryptographic protocol underlying these encrypted conversations.



Why verification? Confrim the security of Signal since it is a secure message app

Signal Shot: A Shared Verification Challenge

Launched in April 2026: a collaborative initiative to formally verify the Signal protocol and application using Lean.

- **Technical ambition:** work toward machine-checked guarantees across the protocol stack and its implementation.
- **Engineering ambition:** develop reusable tools for protocol design and continuous verification as software changes.
- **Community ambition:** bring together Lean, cryptography, software verification, and AI-assisted theorem proving.
- **Concrete workstreams:** the initiative links verification repositories for libsignal, PQXDH, and SPQR.

Roadmap

1. Background: Beneficial AI Foundation and Signal Shot
2. **Practice**
3. Lessons from Practice
4. Prospects

SPQR: A Key-Updating Component of Signal

SPQR = Sparse Post-Quantum Ratchet

- **The protocol:** a component of Signal that updates encryption keys during a conversation, designed to resist quantum attacks.
- **The code:** Signal also provides a Rust library implementing SPQR. This is the software we study.
- **Why "sparse"?** The data needed for a fresh post-quantum key update is spread across multiple messages.

Intuition: Alice and Bob keep updating their keys as they exchange messages.

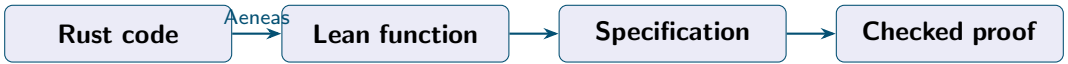
What We Verify

Using Lean, we prove properties of the **Rust code**: for example, that data is encoded correctly and key-update steps follow their specifications. This is implementation correctness, not a proof of all of Signal's security.

Sources: Signal: SPQR overview; SPQR Rust implementation.

SPQR: From Rust Code to Checked Proofs

Target: functional correctness of Signal's Sparse Post-Quantum Ratchet.



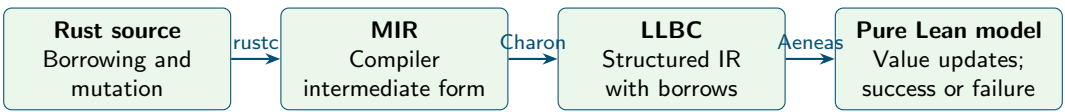
1. **Translate:** use Aeneas to turn the Rust implementation into Lean functions.
2. **Specify:** state what those functions should do, including their assumptions.
3. **Prove:** construct proofs that the translated functions satisfy these specifications; Lean checks the proofs.

The Key Distinction

Translation gives us code we can reason about in Lean. Specifications and proofs establish the properties we want to verify.

Aeneas: Translating Rust into Lean

Inside the translation: Rust control flow becomes a functional model



- **Translation:** Aeneas symbolically executes LLBC, tracking values, borrows, and loans. Mutation becomes explicit value updates in pure functions.
- **Semantic foundation:** formal results relate the borrow-based semantics to a heap-and-address model for the formalized language, not the entire Rust toolchain.

SPQR Example: A Byte Comparison Loop

Compare two byte slices by accumulating differences with bitwise XOR and OR.

Rust: Mutation and a `for` Loop

```
fn compare(lhs: &[u8], rhs: &[u8]) -> u8 {
    let mut r: u8 = 0;
    for i in 0..lhs.len() {
        r |= lhs[i] ^ rhs[i];
    }
    is_non_zero(r)
}
```

`is_non_zero(r)` returns 0 if $r = 0$, and 1 otherwise.

Lean: Explicit State and `Result`

```
def compare (lhs rhs : Slice U8) : Result U8 := do
  let n := Slice.len lhs
  let r ← compare_loop
    { start := 0#usize, «end» := n } lhs rhs 0#u8
  is_non_zero r
```

Loop body at index `i` (names shortened):

```
let a ← Slice.index_usize lhs i
let b ← Slice.index_usize rhs i
let d ← lift (a ^^^ b)
let r1 ← lift (r ||| d)
ok (cont (iter1, r1))
```

Rust construct	**Lean representation**
`&[u8]`	`Slice U8`: a borrowed byte slice becomes a value.
`let mut r; for`	`loop` threads state `(iter, r)`; `cont` continues, `done` exits.
`lhs[i]`	`Slice.index_usize`: a checked access in `Result`; failure propagates.

SPQR Example: Specification and Proof

Specification and Top-Level Proof (compare_spec)

```
theorem compare_spec (lhs rhs : Slice U8)
    (hlen : lhs.length = rhs.length) :
    compare lhs rhs {{ (result : U8) ⇒
      result.val =
        if (∀ (j : Nat), j < lhs.length → lhs.val[j]! = rhs.val[j]!)
        then 0 else 1 }} := by
  unfold compare
  step*
  grind
```

Meaning: equal-length inputs ⇒ successful execution; return 0 for equal slices and 1 otherwise.

Loop invariant: $r = 0 \iff \forall j < i,\ lhs[j] = rhs[j]$.

Initialize $i = 0$, $r = 0$: the empty prefixes match.

Preserve OR accumulates XOR differences; bit-vector lemmas establish the invariant at $i + 1$.

Finish $n - i$ decreases; equal lengths ensure valid accesses. At $i = n$, use `is_non_zero`.

`step*` applies helper specifications; `grind` closes the logical goal. The theorem establishes functional behavior, not constant-time execution.

SPQR: Reviewing Long Specifications

Example: `encaps1` returns ciphertext, intermediate state, and a shared secret.

Contract Sketch (Simplified from Lean)

```
Assume: header length >= 64
  fill_bytes succeeds
Guarantee: there exists one message m
  of 32 bytes such that
  ct1 = ct1List(header, m)
  es  = stateOut(header, m)
  ss  = ssOut(header, m)
Lengths:
  ct1: 960, es: 2080, ss: 32
```

What must a reviewer check?

- All three outputs use the *same* message `m`.
- The named model functions describe the intended bytes, not just their sizes.
- Input conditions match actual callers.
- Success of `fill_bytes` does not establish uniform randomness.

Difficulty: Lean checks the proof of the statement; humans must still check that the statement captures the intended behavior.

Source: `Spqr/Specs/IncrementalMlkem768/Encaps1.lean`, `encaps1_spec`.

SPQR: A True Specification Can Be Too Weak

Example: garbage collection of stored message keys. Old keys should be removed while the required live keys are preserved.

A Plausible but Insufficient Specification

"Every key remaining after cleanup is valid."

Counterexample A: Delete Everything

The output is empty, so every remaining key is valid. The statement holds even though all live keys are lost.

Counterexample B: Duplicate a Live Key

Keep one live key twice and lose another. Each remaining entry can still be valid.

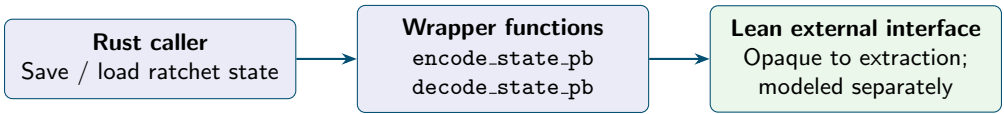
Review obligation: specify which entries survive and preserve their multiplicity, rather than only checking each output entry in isolation.

Lesson: a completed proof cannot repair a missing requirement.

Source: documented review examples in `doc/AI_PR_REVIEW_QUESTIONS.md`.

SPQR: Code Outside the Translation

Example: serializing the ratchet state with Protobuf (`prost`). The project's extraction patch records problems translating the generated `Message` implementations.



- The extraction patch routes calls through wrappers and marks them opaque.
- This lets the surrounding code be translated without translating the `prost` implementation.
- The extraction configuration also excludes platform-specific SIMD and some generated trait implementations.

The Extra Verification Obligation

A proof about a handwritten Lean model still needs justification that the external Rust code behaves like that model.

Sources: `Cargo.toml`; `translation-patches/lib-state_pb.patch`.

SPQR: Reviewing External Assumptions

Example: the external `libcrux_hmac::hmac` call.

Two Different Declarations in Lean

```
noncomputable opaque libcrux_hmac.hmac : ...
axiom libcrux_hmac.hmac_sha256_tag32_spec ...
```

Signatures shortened: the external names a function; the axiom assumes its behavior.

What This Axiom Actually Says

For key and data lengths $\leq$ `u32::MAX`, the SHA-256 HMAC call requesting 32 bytes succeeds and returns 32 bytes.

- **Bounds:** external length conversions to `u32` can panic.
- **Scope:** length and success, not the HMAC value or cryptographic security.
- **Trust:** Lean uses this assumption; downstream proofs depend on it.

Source: `SrcTranslated/FunsExternal.lean`, `hmac_sha256_tag32_spec`.

Dalek: What It Does and What We Verify

- **What is it?** `curve25519-dalek` is a Rust library for group operations on Curve25519 (Edwards and Montgomery forms) and Ristretto.
- **What does it compute?** Point addition $P + Q$, scalar multiplication $[s]P$, and conversions between points, scalars, and bytes.
- **Where is it used?** These operations support key agreement, digital signatures, and zero-knowledge proofs; protocol libraries build on them.

Our Verification Project

Use Aeneas and Lean to verify selected functions in the 64-bit serial backend: field arithmetic modulo $2^{255} - 19$, byte encodings, and curve operations.

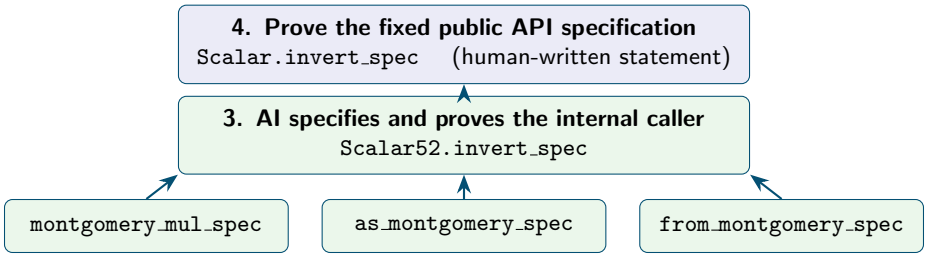
Example guarantee: for a valid point P and scalar bytes with the top bit clear, scalar multiplication succeeds and returns a valid point Q with

$$\text{toPoint}(Q) = [\text{decode}(s)]\,\text{toPoint}(P).$$

This connects the concrete Rust computation to the mathematical group operation.

Auto-Specifying Dalek: Bottom-Up Proofs

1. Human provides and fixes the public API specification.
Implementation code stays fixed; AI writes internal specifications and proofs.

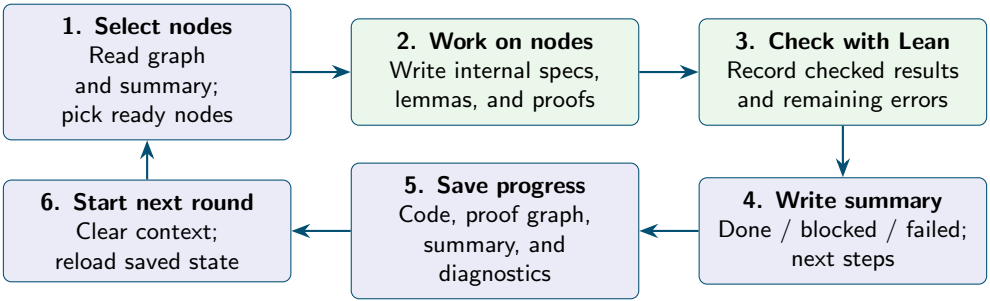


2. AI writes and proves lower-level specs (Scalar52 helpers)

Arrows: proved dependencies support the caller above; continue from the leaves to the API.
Feedback: if an internal spec is too weak, revise it and recheck affected proofs.
Final check: Lean proofs and project gates pass; the supplied API statement stays unchanged.

Harness: Node Selection and Fresh Rounds

Main agent: decide which proof nodes to work on using the dependency graph.



Progress Lives in Files, Beyond the Conversation

Each round starts fresh, but retains the fixed API goal, code, dependency graph, and concise summaries. Repeat until the API proof and final checks pass.

Round design shown here: reset after every summary. The harness also supports session continuation.

Roadmap

1. Background: Beneficial AI Foundation and Signal Shot
2. Practice
3. **Lessons from Practice**
4. Prospects

Lessons from Practice

- AI capabilities in constructing Lean proofs have improved substantially.
- Simple prompts appear sufficient in most cases.
- The clarity of AI-generated explanations has become a key bottleneck.
- A simple design is sufficient for the agent harness.
- Cross-model review is crucial when writing specifications.

Advances in AI-Assisted Lean Proofs

- One year ago, AI struggled to construct challenging proofs.
- At that time, proofs involving loop invariants were beyond its capabilities in my experience.
- In my recent work, I have not encountered a proof that AI could not complete.
- In most cases, the specifications written by AI is correct.

Simple Prompts Often Suffice

- One year ago, Claude Code skills were developed to guide AI workflows.
- These skills use Markdown documents to provide instructions to AI agents.
- The Model Context Protocol (MCP) provides an additional mechanism for tool integration.
- Examples include Lean 4 skills and Lean MCP integrations.

Simple Prompts Often Suffice

- Current AI systems appear substantially more capable.
- I examined how Claude Code operates in my Lean workflow.
- In the sessions I examined, it used neither skills nor MCP tools to construct Lean proofs.
- Why might MCP tools be unnecessary in this workflow?
- MCP tools can expose the tree structure of proofs.
- However, this view may omit global information, such as the number of remaining `sorry` placeholders.
- Given current AI capabilities in Lean, elaborate prompts appear to offer limited additional benefit.
- The most useful instruction appears to be: "Decompose the proof and compile after each completed step."
- Claude Code skills appear less useful in this workflow than they once were.

Clarity of AI-Generated Explanations

- Despite its capabilities, AI still requires human guidance.
- Clear, comprehensible explanations are essential for effective human oversight.
- AI systems tend to produce lengthy responses.
- My most frequent instruction is: "Please shorten your response while preserving precision and clarity."
- AI models should communicate their results in language that users can readily understand.

A Simple Harness Design Is Sufficient

- Use a simple workflow to coordinate proof development.
- Assign a central agent to determine which proof node to expand next.
- Determine which proof nodes can be addressed within the current iteration.
- Record progress after each iteration.
- Reset the working context periodically to prevent excessive accumulation.

A Simple Harness Design Is Sufficient

- Initially, I was unable to complete the proof.
- The `to_bytes` implementation exceeds 700 lines of code, and proof attempts frequently time out.
- The only effective workflow I found was to decompose complex theorems into simpler lemmas.
- Compile after each incremental improvement.

Cross-Model Review for Specification Writing

- AI can generate specifications, but these specifications may contain errors.
- Complement manual review with independent reviews by different AI models.
- This approach is particularly useful for less critical specifications.
- Examples include helper lemmas and functions outside the public API.
- Cross-model review is a practical approach to checking these specifications.

Roadmap

1. Background: Beneficial AI Foundation and Signal Shot
2. Practice
3. Lessons from Practice
4. **Prospects**

Prospects: Important Directions of AI for formal verification

- Efficient Inference
- Understandable Formal verification

We are hiring

Beneficial AI Foundation



Scan to explore open positions

beneficialaifoundation.org/jobs